



Java Collections Framework

تخيل إنك محتاج تحفظ بيانات كثير في برنامجك، زي أسماء الطلاب في فصل، أو درجاتهم، أو حتى علاقة بين الطالب والدرجة بتاعته.

إطار العمل

إيه هو Java Collections Framework؟

هو عبارة عن مجموعة من الـ Classes والـ Interfaces الجاهزة، شغالة ومختبرة، بتوفر لنا هياكل بيانات (Data Structures) مختلفة.

طريقة الاختيار: بنختار الهيكل المناسب حسب احتياجنا:

- ✓ هل محتاج ترتيب؟
- ✓ هل مسموح بالتكرار؟
- ✓ هل محتاج سرعة في البحث؟

مصفوفة تقليدية

مشكلة المصفوفات (Array)

طبعاً تقدر تعمل Array (مصفوفة)، بس المصفوفة جامدة شوية:

- ✓ حجمها ثابت لا يمكن تغييره بعد الإنشاء.
- ✓ مفيش فيها دوال كثير جاهزة للتعامل مع البيانات.

أولاً: عيلة ال List

ال List هي أبسط وأشهر هيكل بيانات. فكر فيها زي "قائمة" أو "لسته" مرتبة. أهم حاجة فيها:

الترتيب	التكرار	الوصول بال Index
↓	📄	#
بتخلي البيانات بالترتيب اللي إنتا دخلتها بيه.	مسموح إنك تدخل نفس القيمة أكثر من مرة.	تقدر توصل لأي عنصر باستخدام رقمه (زي المصفوفة بالضبط).

هنلاقي تحت العيلة دي أربع أنواع رئيسية: Stack, ArrayList, LinkedList, Vector.

ArrayList 1

إمتى نستخدمها؟

- ✓ لما بنكون محتاجين نخزن بيانات و نقرأها كثير.
- ✓ لما بنضيف بيانات في النهاية غالباً.
- ✓ لما مش محتاجين نعمل إضافات وحذف متكررة من البداية أو النص.

أهم Methods:

`get(index)` `add(index, element)` `add(element)`
`clear()` `contains(element)` `size()` `remove(index)`

الشرح والفكرة

دي أسهل نوع وأكثرهم استخداماً. فكر فيها زي المصفوفة (Array) الديناميكية. يعني هي Array لكن حجمها مش ثابت، بتكبر وتصغر أوتوماتيكياً مع إضافة وحذف العناصر.

الخصائص

- ✓ الوصول للعناصر بال Index سريع جداً ($O(1)$).
- ✓ إضافة أو حذف عنصر من النهاية سريع جداً.
- ✓ إضافة أو حذف عنصر من البداية أو النص بطيء شوية (بتنقل العناصر).

ArrayList - Application Example </>

```
import java.util.ArrayList;

public class StudentManager {
    public static void main(String[] args) {
        // عشان نحفظ أسماء الطلاب بنعمل ArrayList
        ArrayList students = new ArrayList<>();

        students.add("محمد");
        students.add("أحمد");
        students.add("سارة");
        System.out.println("الطلاب: " + students);

        students.add(1, "ليلى"); // إضافة طالب في مكان معين (مكان 1)
        String firstStudent = students.get(0); // جلب طالب بال Index
        students.remove("أحمد"); // حذف طالب

        System.out.println("عدد الطلاب: " + students.size());

        for (String name : students) {
            System.out.println("- " + name);
        }
    }
}
```


LinkedList 2

الشرح والفكرة

فكر فيها زي "قطار" أو "سلسلة". كل عنصر (عربية) فيها بيشير للعنصر اللي قبله واللي بعده. مش بتخزن البيانات في مكان واحد متصل زي الـ Array، لا، كل عنصر عبارة عن "Node" (عقدة) لها جزء للبيانات وجزئين للـ Pointers.

الخصائص

- ✓ إضافة وحذف عنصر من البداية أو النص سريع جداً ($O(1)$).
- ✓ الوصول للعناصر بالـ Index بطيء نسبياً ($O(n)$).

إمتى نستخدمها؟

- ✓ لما بنضيف ونشيل كثير من البداية أو النص.
- ✓ لما بنكون بنعمل Stack أو Queue (طواير).
- ✓ لما مش محتاجين الوصول السريع بالـ Index.

أهم Methods الإضافية:

`getLast()``getFirst()``addLast(e)``addFirst(e)``removeLast()``removeFirst()`

LinkedList Example & 3. Vector

:LinkedList (TaskQueue Example)

Synchronized

3. Vector

الشرح والفكرة: قديم جداً (من أيام Java 1.0) ويشبه الـ ArrayList جداً. الفرق الأساسي إن الـ Vector متزامن (Synchronized)، يعني آمن للاستخدام في الـ Multi-threading.

الخصائص: أبطأ من الـ ArrayList في البيئات العادية (Single-thread).
إمتى نستخدمها؟ في التطبيقات القديمة جداً.

نصيحة: في أغلب الأوقات، استخدم ArrayList بدل الـ Vector.

```
import java.util.LinkedList;
import java.util.Queue;

public class TaskQueue {
    public static void main(String[] args) {
        Queue taskQueue = new LinkedList<>();

        taskQueue.add("المهمة الأولى: شغل السيرفر");
        taskQueue.add("المهمة الثانية: تحديث قاعدة البيانات");

        String currentTask = taskQueue.poll(); // FIFO
        String nextTask = taskQueue.peek();
    }
}
```


:Application Example (BrowserHistory)

```
import java.util.Stack;

public class BrowserHistory {
    public static void main(String[] args) {
        Stack history = new Stack<>();

        history.push("google.com");
        history.push("youtube.com");
        history.push("github.com");

        String lastPage = history.pop(); // رجوع
        String currentPage = history.peek(); // الصفحة الحالية
    }
}
```

الشرح والفكرة

الـ Stack عبارة عن "كومة" أو "رزمة". زي رزمة الكتب، إنتا بتضيف كتاب فوق بعض، ولما تجي تاخذ كتاب بتاخذ اللي في الآخر (الي فوق). المبدأ ده اسمه **LIFO (Last In First Out)**.

إمتى نستخدمها؟

- ✓ في خوارزميات الـ Parsing (التحقق من الأقواس).
- ✓ في الـ Undo/Redo (التراجع وإعادة العمل).
- ✓ في خوارزميات الـ DFS (Depth First Search).

أهم Methods:

empty() peek() pop() push(element)

ثانياً: عيلة ال Set

ال Set زي المجموعة في الرياضيات: مفيش فيها تكرار، وترتيبها مش مهم (في معظم أنواعها).

1. HashSet

الشرح: بتستخدم ال Hashing (دالة تجزئة).

الخصائص: أسرع نوع، مفيش أي ترتيب للعناصر (Random Order)، وتسمح بوضع null (مرة واحدة).

إمتى نستخدمها؟ لمنع التكرار وبسرعة عالية في البحث (contains).

2. LinkedHashSet

الشرح: زي ال HashSet بالضبط، لكن مضاف إليه LinkedList.

الخصائص: بيحافظ على ترتيب الإدراج (Insertion Order).

إمتى نستخدمها؟ لما نكون عايزين نمنع التكرار ونحافظ على ترتيب الإدخال.

3. TreeSet

الشرح: بتخزن العناصر في شجرة (Tree).

الخصائص: بترتب العناصر أوتوماتيكياً (تصاعدياً/تنازلياً)، أبطأ شوية ($O(\log n)$)، ومشي بتسمح بـ null.

إمتى نستخدمها؟ لمنع التكرار والحصول على عناصر مرتبة (Sorted).

Set - Application Examples </>

:TreeSet (Sorted Scores)

```
TreeSet scores = new TreeSet<>();
scores.add(85);
scores.add(70);
scores.add(95); // مرتبة تصاعدياً

System.out.println("أقل درجة: " + scores.first());
System.out.println("أعلى درجة: " + scores.last());
```

:LinkedHashSet (Playlist)

```
LinkedHashSet playlist = new LinkedHashSet<>();
playlist.add("1 أغنية");
playlist.add("2 أغنية");
playlist.add("1 أغنية"); // لن تتكرر

// تظهر بترتيب الإضافة
System.out.println(playlist);
```

:HashSet (Unique Visitors)

```
HashSet visitors = new HashSet<>();
visitors.add("Ahmed");
visitors.add("Sara");
visitors.add("Ahmed"); // يتجاهلها

if (visitors.contains("Sara")) {
    System.out.println("سارة موجودة");
}
```


ثالثاً وأهم حاجة: عيلة ال Map

ال Map مش Set ولا List. دي بتخزن البيانات في شكل أزواج (**Key-Value**). كل Key بيشار على Value. فكر فيها زي القاموس (Dictionary).

القيمة (Value)

ممکن تتكرر عادي مع مفاتيح مختلفة.

المفتاح (Key) فريد

مينفعش يتكرر، وأي قيمة جديدة لنفس المفتاح بتستبدل القديمة.

الأكثر استخداماً

1. HashMap

الشرح والخصائص: بتستخدم ال Hashing، أسرع نوع من ال Maps، مفيش ترتيب معين للأزواج، وبتسمح بوضع $Key = null$ (مرة واحدة) و $Values = null$ (كثير).
إمتى نستخدمها؟ في 90% من الحالات اللي محتاجين فيها Map لربط بيانات ببعض والبحث السريع.

`entrySet()``values()``keySet()``containsKey(key)``get(key)``put(key, value)`

🔗 أنواع الـ Map الأخرى

2. LinkedHashMap

الشرح: زي الـ HashMap، مع LinkedList للحفاظ على الترتيب.

الخصائص: ييحافظ على ترتيب الإدراج (Insertion Order).

إمتى نستخدمها؟ لما نكون عايزين مزايا الـ HashMap ومهم الترتيب.

3. TreeMap

الشرح: بتخزن الأزواج في شجرة (Tree) مرتبة حسب الـ Key.

الخصائص: أبطأ شوية ($O(\log n)$)، المفاتيح مرتبة، ولا تسمح بـ null Key.

إمتى نستخدمها؟ لما نحتاج Map بمفاتيح مرتبة.

4. Hashtable

Legacy

الشرح: الجدة القديمة للـ HashMap (من Java 1.0).

الخصائص: متزامنة (Synchronized)، أبطأ، ولا تسمح بـ null في Key أو Value.

إمتى نستخدمها؟ مبنستخدامهاش في الكود الحديث، بنفضل ConcurrentHashMap.